SOFTWARE REQUIREMENTS SPECIFICATION

ShieldDrive

Vehicle Insurance Management System (VIMS)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Vehicle Insurance Management System, commercially named ShieldDrive. It serves as the authoritative reference for developers, testers, architects, and stakeholders throughout the project lifecycle.

The document captures the system's intended behavior, constraints, user roles, business logic engines, data design, and interface specifications, ensuring all parties share a single, unambiguous understanding of what the system must do and how it must perform.

1.2 Project Overview

ShieldDrive (VIMS) is a full-stack, enterprise-grade digital platform designed to modernize and automate the entire vehicle insurance lifecycle. Historically, the insurance sector has relied on manual data entry, physical document handling, and fragmented workflows, which led to delayed claim settlements, opaque status tracking, and high administrative costs.

VIMS replaces this with a centralized, role-driven digital infrastructure covering customer onboarding, vehicle registration, policy issuance, dynamic premium calculation, multi-stage claim processing, and payment management, all delivered through a secure RESTful API consumed by a modern Angular frontend.

1.3 Scope

The system covers the following functional domains:

- User registration, authentication, and role-based access control
- Vehicle registration and management per customer
- Insurance policy plan administration by admins
- Policy purchasing, underwriter review, and lifecycle tracking
- Dynamic premium calculation using vehicle risk and IDV algorithms
- Multi-stage claim submission, survey, and financial adjudication
- Payment processing and transaction history
- Document upload for claims evidence
- Admin analytics dashboard with revenue and policy metrics

Out of scope for version 1.0: real payment gateway integration, RTO/DMV API connectivity, AI-based damage prediction, and SMS/email notification services.

1.4 Definitions and Abbreviations

Term / Abbreviation	Definition
SRS	Software Requirements Specification
VIMS	Vehicle Insurance Management System
IDV	Insured Declared Value. This is 80% of the vehicle's current market value
NCB	No Claim Bonus. A discount applied to the premium for vehicles with no prior claims
RBAC	Role-Based Access Control
JWT	JSON Web Token. The stateless authentication mechanism used by the system
ORM	Object-Relational Mapper (Entity Framework Core)
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
SPA	Single Page Application
3NF	Third Normal Form. The database normalization standard used in VIMS
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
PDF	Portable Document Format
EF Core	Entity Framework Core, the .NET ORM used for data access

1.5 Document Conventions

- Requirements are labeled FR-XX (functional) and NFR-XX (non-functional)
- All monetary values are in USD unless stated otherwise
- "User" refers to any authenticated entity; role-specific terms (Customer, Agent, Admin, etc.) are used where precision is needed
- "System" refers to the complete VIMS application including the frontend, backend, and database

1.6 References

- IEEE Std 830-1998: IEEE Recommended Practice for SRS
- ASP.NET Core Documentation by Microsoft
- Angular Documentation by Google
- iText 9 Documentation by iText Group

2. Overall Description

2.1 Product Perspective

VIMS is a standalone, full-stack web application built on a decoupled client-server architecture. The Angular frontend communicates exclusively with the ASP.NET Core backend via REST APIs secured by JWT. The system integrates no external services in v1.0. All payment, notification, and document generation logic is self-contained.

The system is designed to be deployable on-premise or in cloud environments (Azure, AWS) with minimal configuration changes, using environment variables for connection strings and secrets.

2.2 Product Functions (Summary)

#	Function	Primary Actor(s)
1	User registration and secure login	All roles
2	Vehicle registration and management	Customer
3	Policy plan creation and administration	Admin
4	Policy purchase and activation	Customer, Agent
5	Dynamic premium quotation	Customer, Agent
6	Underwriter risk review and policy approval	Underwriter
7	Claim submission with document upload	Customer
8	Field survey and report submission	Surveyor
9	Claim financial adjudication	Claims Officer
10	Payment processing and history	Customer
11	PDF policy document generation	System
12	Admin analytics dashboard	Admin
13	System settings management	Admin

2.3 User Classes and Characteristics

Role	Description	Technical Expertise
Customer	Primary end-user. Registers vehicles, buys policies, files claims, makes payments.	Low. Consumer-friendly UI required
Agent	Sales intermediary. Submits policies on behalf of customers, tracks portfolio.	Medium. Internal staff
Underwriter	Risk assessor. Reviews high-risk policy applications and approves or rejects them.	High. Insurance domain expert
Surveyor	Field inspector. Investigates claims, submits damage reports.	Medium. Field technical staff
Claims Officer	Financial adjudicator. Final approval/rejection of claims with coverage enforcement.	High. Financial decision maker
Admin	Super-user. Manages plans, users, settings, and analytics.	High. System administrator

2.4 Operating Environment

Component	Technology / Environment
Backend Framework	ASP.NET Core Web API (.NET 8/9)
Frontend Framework	Angular (TypeScript), Single Page Application
Database	Microsoft SQL Server
ORM	Entity Framework Core with LINQ
Authentication	JSON Web Tokens (JWT) with role claims
PDF Generation	iText 9 + BouncyCastle.Cryptography
API Documentation	Swagger / OpenAPI (Swashbuckle)
UI Styling	Tailwind CSS, Angular Material, Angular CDK
Charts	Chart.js via ng2-charts
Animations	Framer Motion

Component	Technology / Environment
State Management	RxJS
Browsers Supported	Chrome 110+, Firefox 110+, Edge 110+, Safari 16+
Deployment	On-premise or Azure/AWS cloud

2.5 Design and Implementation Constraints

- All API endpoints (except auth) must require a valid JWT token
- Passwords must be hashed before storage. Plain text storage is prohibited
- Claim amounts must never exceed the policy's remaining coverage limit
- The system must enforce state-machine transitions. Invalid status changes are blocked at the service layer
- File uploads for claim documents must be stored on the filesystem with only paths saved in the database
- All monetary calculations must use decimal types to prevent floating-point errors

2.6 Assumptions and Dependencies

- SQL Server is available and accessible by the backend service
- The iText 9 library license is valid for production use
- All users have access to a modern web browser
- The system runs on HTTPS in production. JWT tokens must not be transmitted over plain HTTP
- Payment processing is simulated in v1.0. No real financial transactions occur

3. System Architecture

3.1 Architectural Overview

VIMS is built on a decoupled, multi-tiered RESTful architecture. Each layer has a single, well-defined responsibility, promoting separation of concerns, testability, and maintainability.

Layer	Technology	Responsibility
Presentation	Angular SPA (TypeScript)	Renders role-specific UI, handles input validation, consumes REST APIs asynchronously
API / Routing	ASP.NET Core Controllers	Routes HTTP requests, validates JWT, enforces RBAC via [Authorize] attributes, parses DTOs
Business Logic	Service Layer (C#)	Enforces domain rules: premium calculation, state machine transitions, coverage limit checks
Data Access	Repository Pattern + EF Core	Abstracts SQL, manages CRUD operations, handles entity relationships and transactions
Storage	Microsoft SQL Server	Persists all structured data including users, vehicles, policies, claims, payments, and documents

3.2 Interaction Flow: Raising a Claim (Example)

```
1. Customer submits claim via Angular UI
2. Angular → POST /api/claims/raise + JWT in Authorization header
3. ASP.NET Controller: validates JWT signature and role
4. Controller passes DTO to ClaimService
5. ClaimService: verifies policy is Active, checks incident date within policy term
6. ClaimService: verifies claimed amount ≤ remaining coverage
7. Repository: inserts Claim record (Status = Pending)
8. Service returns ClaimResponse DTO
9. Controller returns 200 OK → Angular updates UI
```

3.3 Security Architecture

- JWT tokens are signed with a server-side secret key and contain role claims
- All backend controllers apply [Authorize(Roles = "...")] attribute enforcement
- The Angular frontend decodes the JWT payload to render role-appropriate navigation, but backend enforcement is the true security boundary
- Passwords are hashed using a cryptographic algorithm before database storage
- PDF documents are cryptographically signed via iText 9 + BouncyCastle

4. Functional Requirements

4.1 Authentication & User Management

ID	Requirement	Actor	Priority
FR-01	The system shall allow any user to register with fullName, email, password, role, securityQuestion, and securityAnswer.	Public	High
FR-02	The system shall validate that the email address is unique across all accounts.	System	High
FR-03	The system shall hash all passwords before storing them in the database.	System	High
FR-04	The system shall authenticate users via email and password and return a signed JWT containing userId and role.	Public	High
FR-05	The system shall allow users to initiate a password reset via a security question challenge.	Public	Medium
FR-06	The Admin shall be able to force-reset any user's password.	Admin	Medium
FR-07	The system shall allow authenticated users to retrieve their own profile.	All roles	Medium
FR-08	The Admin shall be able to list all registered users.	Admin	Medium

4.2 Vehicle Management

ID	Requirement	Actor	Priority
FR-09	The system shall allow authenticated users to register a vehicle with registrationNumber, make, model, year, chassisNumber, currentMarketValue, and vehicleType.	Customer	High
FR-10	The system shall validate that the registrationNumber is unique across all vehicles.	System	High
FR-11	The system shall allow customers to view all their registered vehicles.	Customer	High
FR-12	The system shall allow customers to update vehicle details.	Customer	Medium
FR-13	The system shall allow customers to delete a vehicle, provided it has no active policy.	Customer	Medium

4.3 Policy Plan Management

ID	Requirement	Actor	Priority
FR-14	The Admin shall be able to create policy plans with planName, description, policyType, basePremium, maxCoverageAmount, and policyDurationMonths.	Admin	High
FR-15	The Admin shall be able to update and deactivate existing policy plans.	Admin	High
FR-16	All authenticated users shall be able to browse the list of active policy plans.	All	High
FR-17	The Admin shall not be able to delete a plan that has active policies attached to it.	Admin	Medium

4.4 Policy Management

ID	Requirement	Actor	Priority
FR-18	The system shall allow customers (or agents on their behalf) to purchase a policy by selecting a vehicle and a plan.	Customer, Agent	High
FR-19	The system shall calculate and store the premium amount at the time of policy creation using the Premium Calculation Engine.	System	High
FR-20	If the vehicle type is flagged as high-risk, the policy shall be routed to the Underwriter queue before activation.	System	High
FR-21	The Underwriter shall be able to approve, reject, or adjust the premium of policies in the pending review queue.	Underwriter	High
FR-22	Upon successful payment, the policy status shall transition to Active with explicit StartDate and EndDate set.	System	High
FR-23	The system shall generate a cryptographically signed PDF of the policy contract upon activation via iText 9.	System	High
FR-24	The Agent shall be able to view all policies assigned to them.	Agent	Medium
FR-25	The system shall block any claim against a policy that is not in Active status.	System	High

4.5 Premium Calculation

ID	Requirement	Actor	Priority
FR-26	The system shall calculate IDV as 80% of the vehicle's currentMarketValue.	System	High
FR-27	The system shall calculate the base premium as 2% of the IDV.	System	High
FR-28	The system shall apply a 5% risk surcharge per risk point from the Vehicle Risk Score Engine.	System	High
FR-29	The system shall apply a +20% old vehicle surcharge for vehicles older than 10 years.	System	High
FR-30	The system shall apply a -10% new vehicle discount for vehicles under 2 years old.	System	High
FR-31	The system shall apply a 10% No Claim Bonus discount if the vehicle's risk score is below 7.	System	High
FR-32	The final premium must not fall below the system minimum floor of 500 currency units.	System	High
FR-33	The system shall expose a read-only premium quote endpoint that calculates and returns a full breakdown without persisting any data.	System	High

4.6 Vehicle Risk Score Engine

The VehicleRiskService computes a standardized risk score (1–10, capped at 10) for each vehicle. This score feeds into both premium calculation and underwriting decisions.

Parameter	Rule	Points
Vehicle Age	Age > 15 years	+3
Vehicle Age	Age > 8 years	+2
Vehicle Age	Age > 3 years	+1
Vehicle Type	Supercar or Luxury	+3
Vehicle Type	SUV	+1
Historical Claims	1 previously approved claim	+2
Historical Claims	More than 1 previously approved claim	+2 additional
Score Cap	Maximum risk score regardless of cumulative points	10

4.7 Claim Management

ID	Requirement	Actor	Priority
FR-34	The Customer shall be able to submit a claim against an Active policy with incidentDate, incidentLocation, description, and claimedAmount.	Customer	High
FR-35	The system shall reject any claim where the incidentDate falls outside the policy's StartDate and EndDate.	System	High
FR-36	The system shall reject any claim where the claimedAmount exceeds the policy's remaining coverage (maxCoverage minus sum of approved payouts).	System	High
FR-37	The Customer shall be able to upload supporting documents (images, PDFs) for their claim.	Customer	High
FR-38	The system shall assign claims to a Surveyor for field inspection.	System / Admin	High
FR-39	The Surveyor shall be able to submit a field report with damage assessment and recommended payout amount.	Surveyor	High
FR-40	The Surveyor's report submission shall transition the claim to 'Awaiting Officer Approval' status.	System	High
FR-41	The Claims Officer shall be able to approve, partially approve, or reject the claim.	Claims Officer	High
FR-42	The Claims Officer's approved amount must not exceed the policy's remaining coverage limit. The system enforces this automatically.	System	High
FR-43	Upon approval, the system shall update remaining coverage by deducting the approved amount.	System	High

4.8 Payment Management

ID	Requirement	Actor	Priority
FR-44	The system shall allow customers to process premium payments via supported methods (CreditCard, DebitCard, NetBanking, UPI, Wallet).	Customer	High
FR-45	The system shall create a Payment record with transactionId, amount, status, and paymentDate upon processing.	System	High
FR-46	Payment processing is asynchronous. Clients must poll the payment endpoint to confirm the final status.	System	High
FR-47	The system shall allow customers to view their full payment history.	Customer	Medium

4.9 Document Management

ID	Requirement	Actor	Priority
FR-48	The system shall accept multipart/form-data file uploads for claim evidence.	Customer	High
FR-49	Uploaded files shall be stored on the filesystem; only the file path shall be saved in the database.	System	High
FR-50	The system shall associate uploaded documents with the correct claim via claimId.	System	High

4.10 Admin Dashboard & Settings

ID	Requirement	Actor	Priority
FR-51	The Admin dashboard shall display TotalUsers, TotalRevenue, TotalPolicies, TotalClaims, MonthlyRevenue, and PolicyStatusDistribution.	Admin	Medium
FR-52	The system shall allow the Admin to bulk-update system settings (e.g. TaxRate, MaxClaimLimit) via an array of key-value pairs.	Admin	Medium

5. Claim Processing Lifecycle

Claims follow a strict state machine. No transition can be skipped or reversed. The system enforces valid transitions at the service layer.

Current Status	Trigger / Action	Next Status	Actor
—	Customer submits claim	Submitted	Customer
Submitted	Admin/System assigns surveyor	Under Survey	Admin / System
Under Survey	Surveyor submits field report	Awaiting Officer	Surveyor
Awaiting Officer	Claims Officer approves claim	Approved	Claims Officer
Awaiting Officer	Claims Officer rejects claim	Rejected	Claims Officer
Awaiting Officer	Claims Officer partially approves	Approved (Partial)	Claims Officer
Approved / Rejected	No further transitions permitted	—	—

6. Non-Functional Requirements

6.1 Security

ID	Requirement
NFR-01	All API endpoints except /auth/register and /auth/login must require a valid JWT Bearer token.
NFR-02	JWT tokens must be signed with a server-managed secret and must include expiry claims.
NFR-03	Passwords must never be stored in plain text. bcrypt or equivalent hashing is required.
NFR-04	Role-based access must be enforced at the API controller level, not just the frontend.
NFR-05	Generated PDF policy documents must be cryptographically signed and tamper-evident.
NFR-06	Uploaded claim documents must be stored outside the web root to prevent direct URL access.

6.2 Performance

ID	Requirement
NFR-07	API response time for standard CRUD endpoints must be under 500ms under normal load.
NFR-08	The premium calculation engine must return a quote within 300ms.
NFR-09	The system must support at least 100 concurrent users without degradation.
NFR-10	File uploads for claim documents must not block the main server thread (streaming required).

6.3 Reliability & Availability

ID	Requirement
NFR-11	The system must target 99.5% uptime during business hours.
NFR-12	Database transactions must be ACID-compliant. Partial writes on failure must be rolled back.
NFR-13	The system must handle malformed JWT tokens gracefully and always return 401, never 500.

6.4 Maintainability & Scalability

ID	Requirement
NFR-14	The backend must follow Clean Architecture. Controllers, Services, and Data Access Layers must be clearly separated.
NFR-15	The database schema must be maintained via EF Core migrations. Manual DDL scripts are not allowed.
NFR-16	The system must be deployable with environment-variable configuration (no hardcoded secrets).
NFR-17	The architecture must support horizontal scaling through a stateless API design using JWT with no server-side sessions.

6.5 Usability

ID	Requirement
NFR-18	The Angular frontend must render role-appropriate navigation based on the JWT payload. Users must never see menus or features they cannot access.
NFR-19	All forms must provide real-time validation feedback before submission.
NFR-20	The UI must be responsive across desktop and tablet screen sizes.
NFR-21	The Swagger UI must be available in development for API testing by developers.

7. Database Design

The relational database is normalized to Third Normal Form (3NF) to ensure data integrity, eliminate redundancy, and support accurate historical tracking.

7.1 Entity Overview

Table	Primary Key	Key Foreign Keys	Description
Users	UserId	—	All system users. Role enum: Admin, Customer, Agent, Underwriter, Surveyor, ClaimsOfficer
Vehicles	VehicleId	CustomerId → Users	Customer-owned vehicles. RegistrationNumber must be unique.
PolicyPlans	PlanId	—	Insurance products defined by Admin. IsActive flag controls visibility.
Policies	PolicyId	CustomerId, AgentId, VehicleId, PlanId → respective tables	Active insurance contracts. PreviousPolicyId enables renewals.
PolicyCoverages	CoverageId	PolicyId → Policies	Coverage types attached to a specific policy.
Claims	ClaimId	PolicyId, CustomerId, ClaimsOfficerId → Users	Insurance claims. ApprovedAmount tracked for coverage saturation.
ClaimDocuments	DocumentId	ClaimId → Claims, UploadedBy → Users	File paths of uploaded claim evidence. Actual files stored on filesystem.
Payments	PaymentId	PolicyId → Policies	Premium payment records. Status: Pending / Paid / Failed.
Settings	SettingKey	—	System-wide configuration key-value pairs managed by Admin.

7.2 Key Relationships

- One Customer → Many Vehicles (1:N)
- One Vehicle → Many Policies (1:N)
- One PolicyPlan → Many Policies (1:N)
- One Policy → Many Claims (1:N)
- One Claim → Many ClaimDocuments (1:N)
- One Policy → One Payment (1:1 per premium period)
- PreviousPolicyId on Policies enables linked renewal chains

8. External Interface Requirements: API

All endpoints communicate over HTTPS using JSON (application/json) unless noted. Every protected endpoint requires: Authorization: Bearer <jwt_token>

8.1 Standard Response Envelope

```
{
  "success": true,
  "message": "Optional description",
  "data": { ... }   // payload, array, or null
}
```

8.2 Endpoint Summary

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, receive JWT
POST	/api/auth/forgot-password	Public	Initiate password reset
POST	/api/auth/reset-password	Public	Submit new password
PUT	/api/auth/admin-reset-password/{userId}	Admin	Force-reset user password
GET	/api/auth/profile	Authenticated	Get own profile
GET	/api/auth/users	Admin	List all users
POST	/api/Vehicle	Authenticated	Register vehicle
GET	/api/Vehicle	Authenticated	List own vehicles
GET	/api/Vehicle/{id}	Authenticated	Get vehicle by ID
PUT	/api/Vehicle/{id}	Authenticated	Update vehicle
DELETE	/api/Vehicle/{id}	Authenticated	Remove vehicle
POST	/api/PolicyPlan	Admin	Create plan
GET	/api/PolicyPlan	Authenticated	List plans
GET	/api/PolicyPlan/{id}	Authenticated	Get plan
PUT	/api/PolicyPlan/{id}	Admin	Update plan
DELETE	/api/PolicyPlan/{id}	Admin	Delete plan
POST	/api/Policy	Authenticated	Purchase policy
GET	/api/Policy	Authenticated	List own policies
GET	/api/Policy/assigned	Agent	List assigned policies
GET	/api/Policy/{id}	Authenticated	Get policy
PUT	/api/Policy/{id}/approve	Agent	Approve / reject policy
GET	/api/Premium	Authenticated	Get premium quote
POST	/api/Claim	Customer	Submit claim
GET	/api/Claim	Customer / CO	List claims
GET	/api/Claim/{id}	Authenticated	Get claim
PUT	/api/Claim/{id}/review	ClaimsOfficer	Adjudicate claim
POST	/api/Payment	Authenticated	Process payment
GET	/api/Payment/my	Authenticated	List own payments
GET	/api/Payment/{id}	Authenticated	Get payment
POST	/api/claim-documents/{claimId}	ClaimsOfficer	Upload claim document
GET	/api/AdminDashboard/stats	Admin	Dashboard statistics
POST	/api/Settings/bulk-update	Admin	Update system settings

8.3 Error Codes

HTTP Code	Meaning	Common Cause
400	Bad Request	Missing required fields, invalid input, or business rule violation
401	Unauthorized	Missing or expired JWT token
403	Forbidden	Valid token but insufficient role permissions
404	Not Found	Requested resource does not exist
409	Conflict	Duplicate record, for example an email or registration number that already exists
422	Unprocessable Entity	Data is valid syntactically but fails business logic checks
500	Internal Server Error	Unexpected server-side failure

9. UML Diagrams

The following diagrams provide a comprehensive visual specification of the VIMS system, covering actors, structure, data, workflows, and interaction sequences.

9.1 Use Case Diagram

Illustrates the interactions between the four primary actors (Claims Officer, Admin, Agent, and Customer) and the functional use cases available to each within the system.

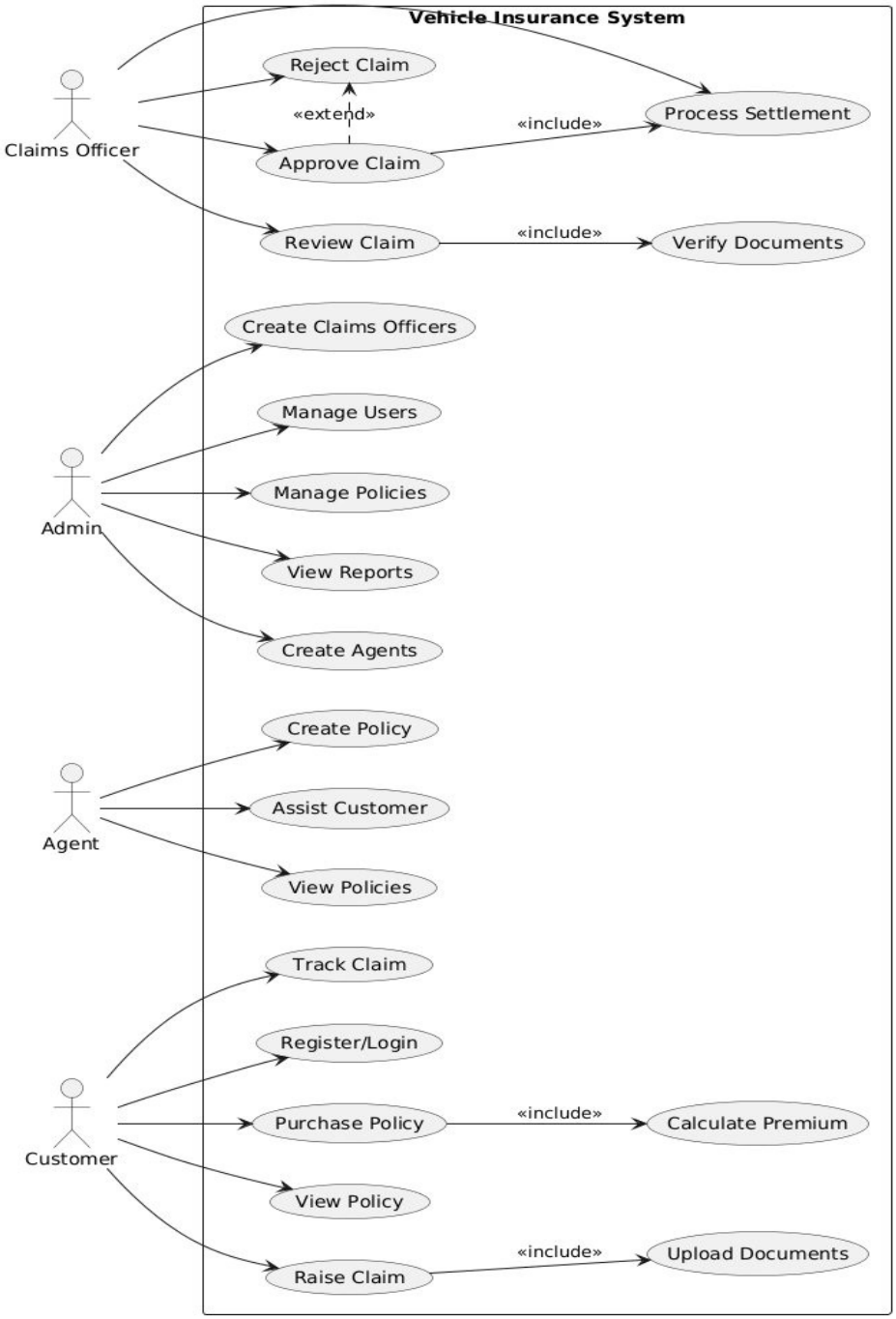

Figure 1: Use Case Diagram for the Vehicle Insurance Management System

9.2 Class Diagram

Depicts the object-oriented domain model including core classes such as User, Vehicle, PolicyPlan, Policy, Claim, Payment, and ClaimDocument, along with their attributes and relationships.

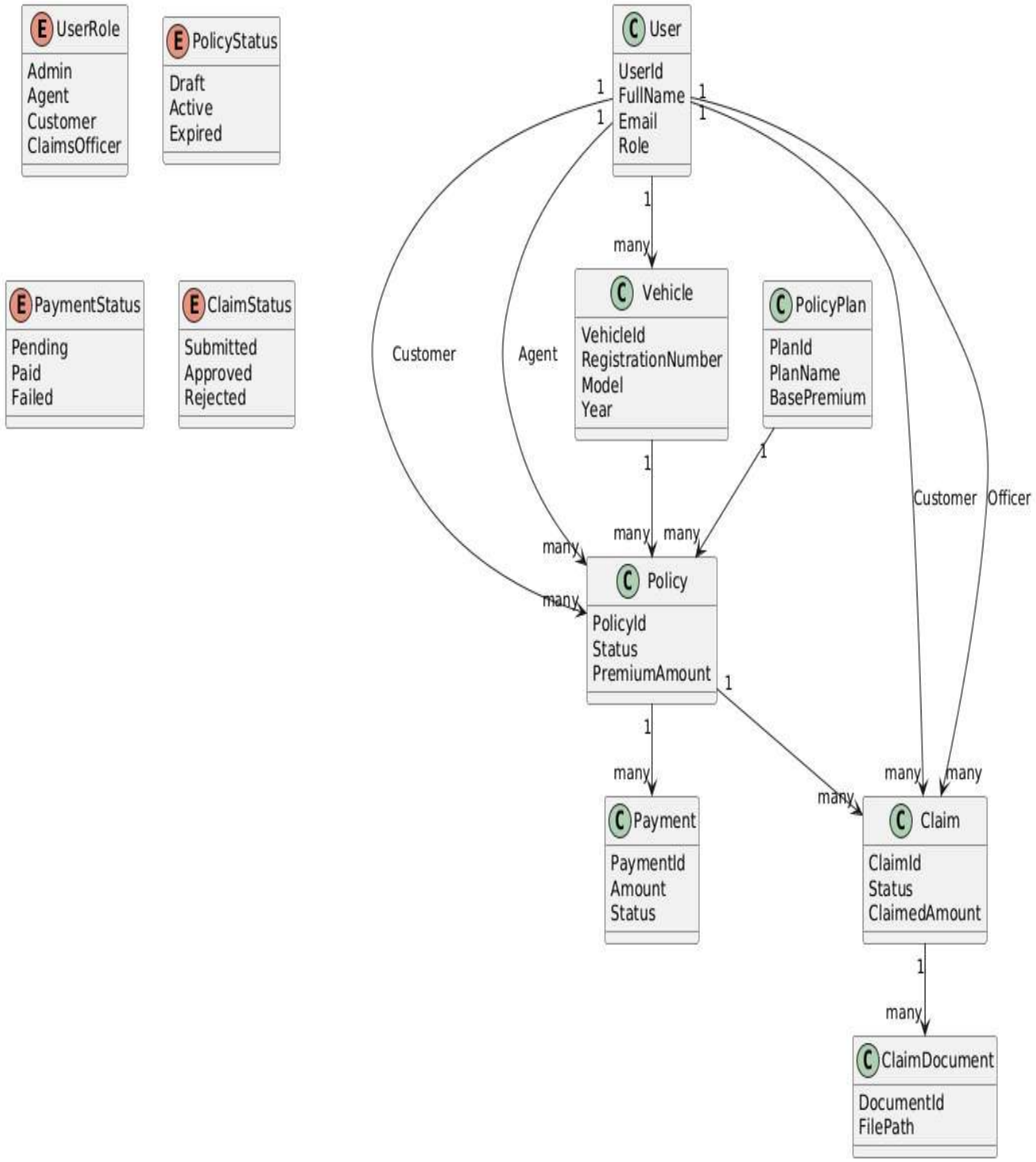

Figure 2: Class Diagram showing the Domain Model

9.3 Entity-Relationship (ER) Diagram

Provides the complete database schema with a normalized relational structure, showing primary keys, foreign keys, data types, and the cardinality of all entity relationships.

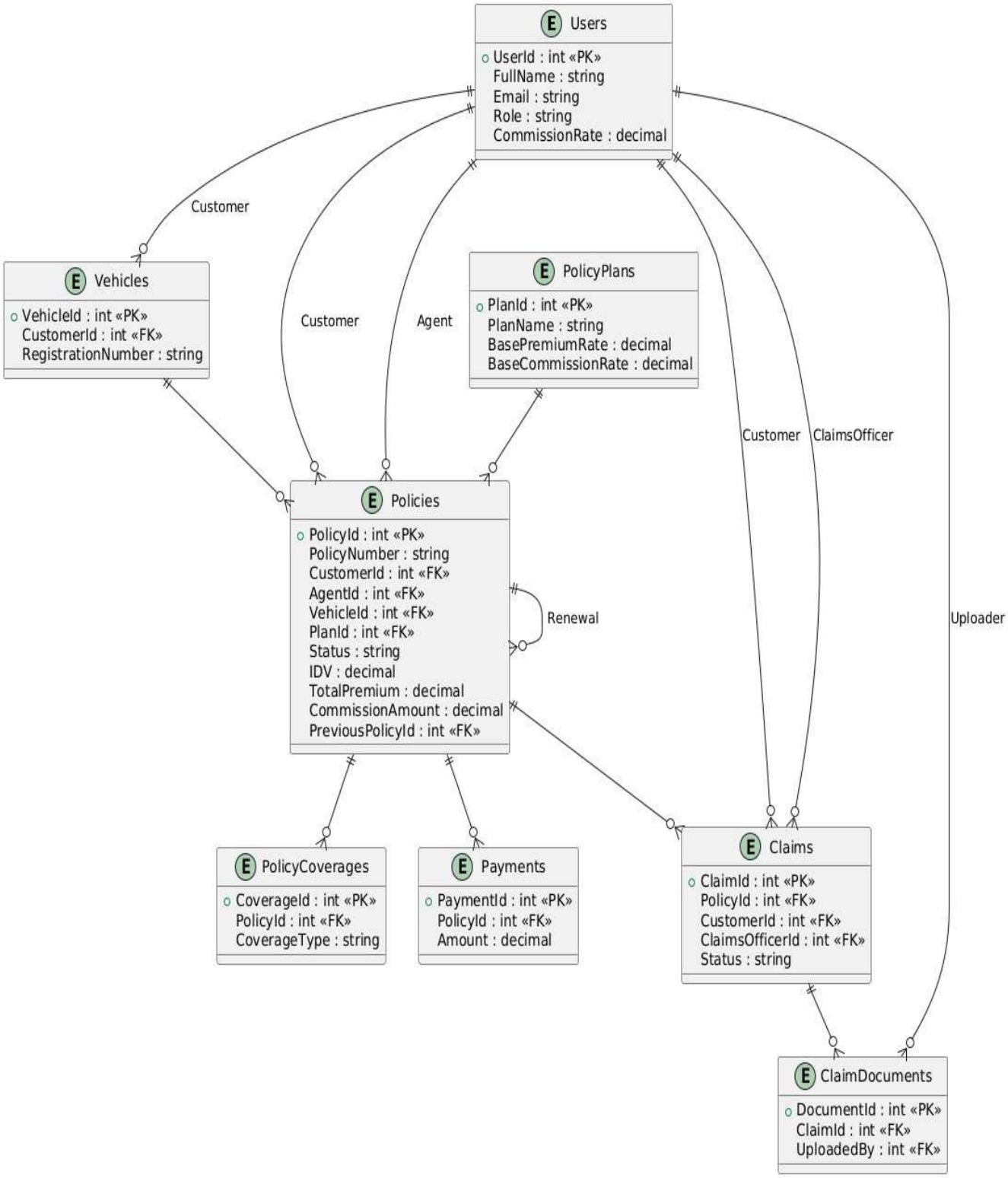

Figure 3: Entity-Relationship Diagram of the Database Schema

9.4 Workflow / Activity Diagram

Traces the end-to-end operational flow for all user roles, from registration and login through role-specific activities for the Customer, Agent, Admin, and Claims Officer.

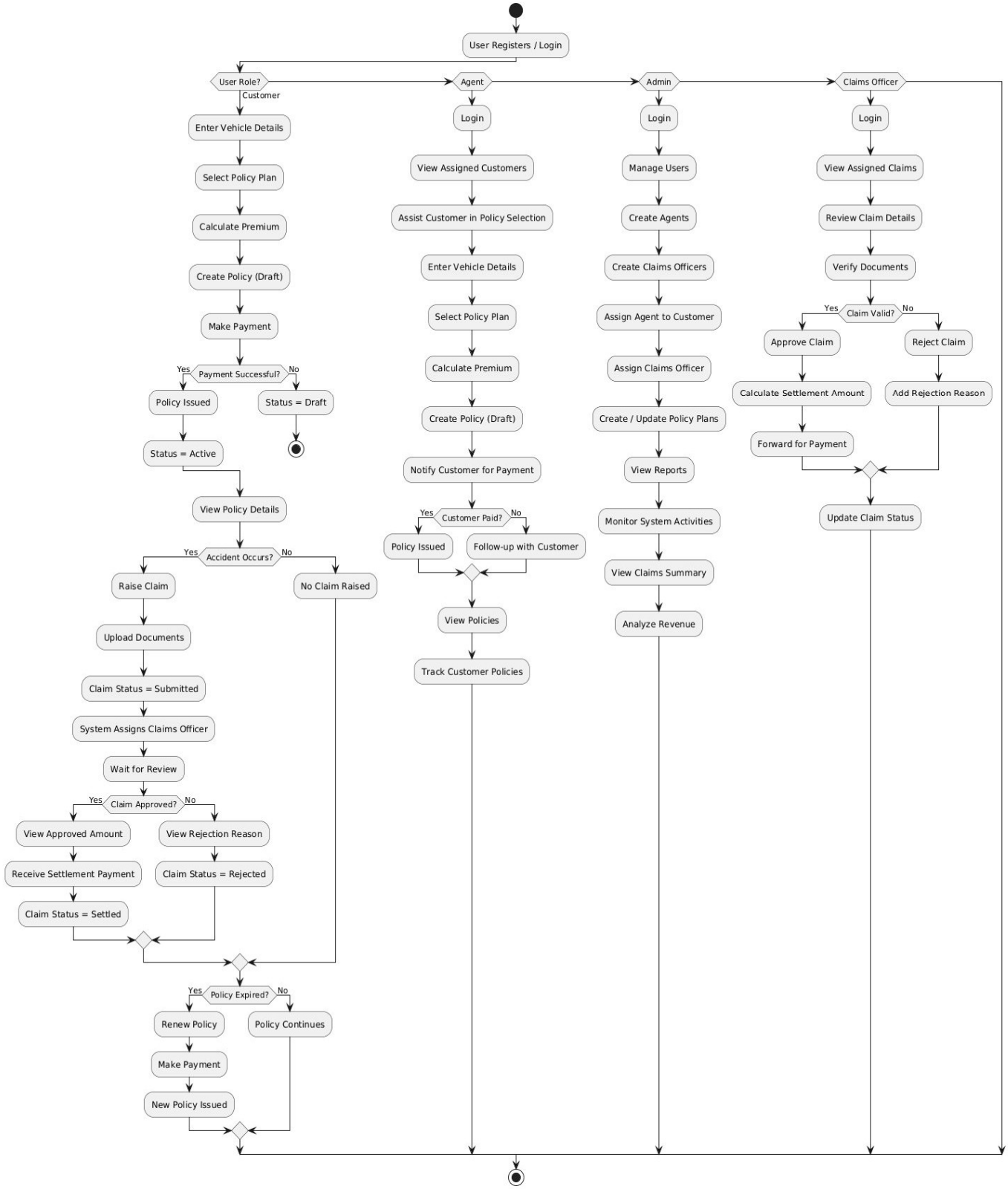

Figure 4: Workflow and Activity Diagram for the Full System Lifecycle

9.5 Policy Sequence Diagram

Details the step-by-step interactions between the Customer, UI, PolicyController, PolicyService, PaymentService, and Database during the policy creation, premium calculation, payment, and activation lifecycle.

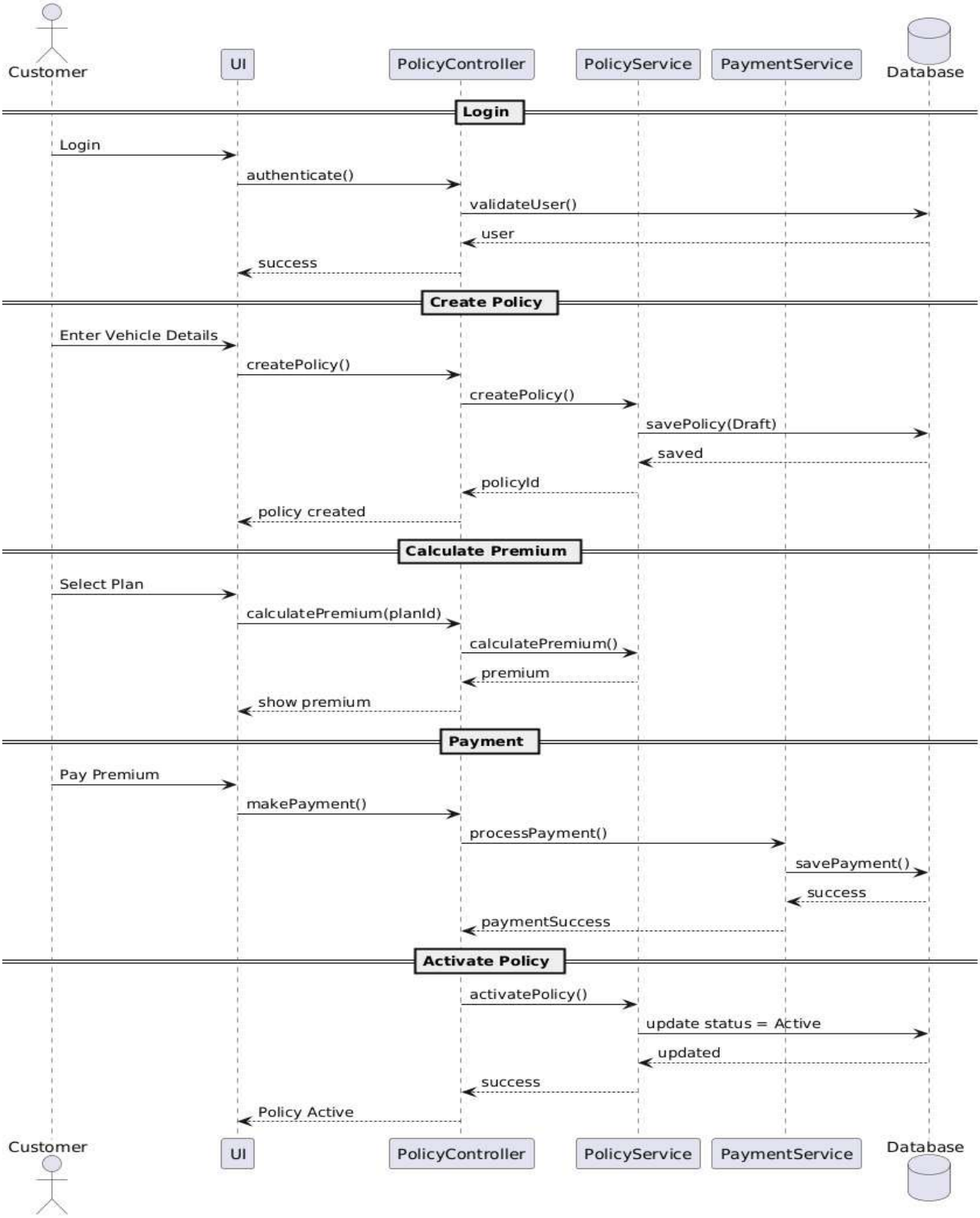

Figure 5: Policy Sequence Diagram showing the Policy Lifecycle Flow

9.6 Claims Sequence Diagram

Illustrates the claims processing pipeline, from claim submission and document upload through Surveyor review, all the way to the Claims Officer's final decision and settlement.

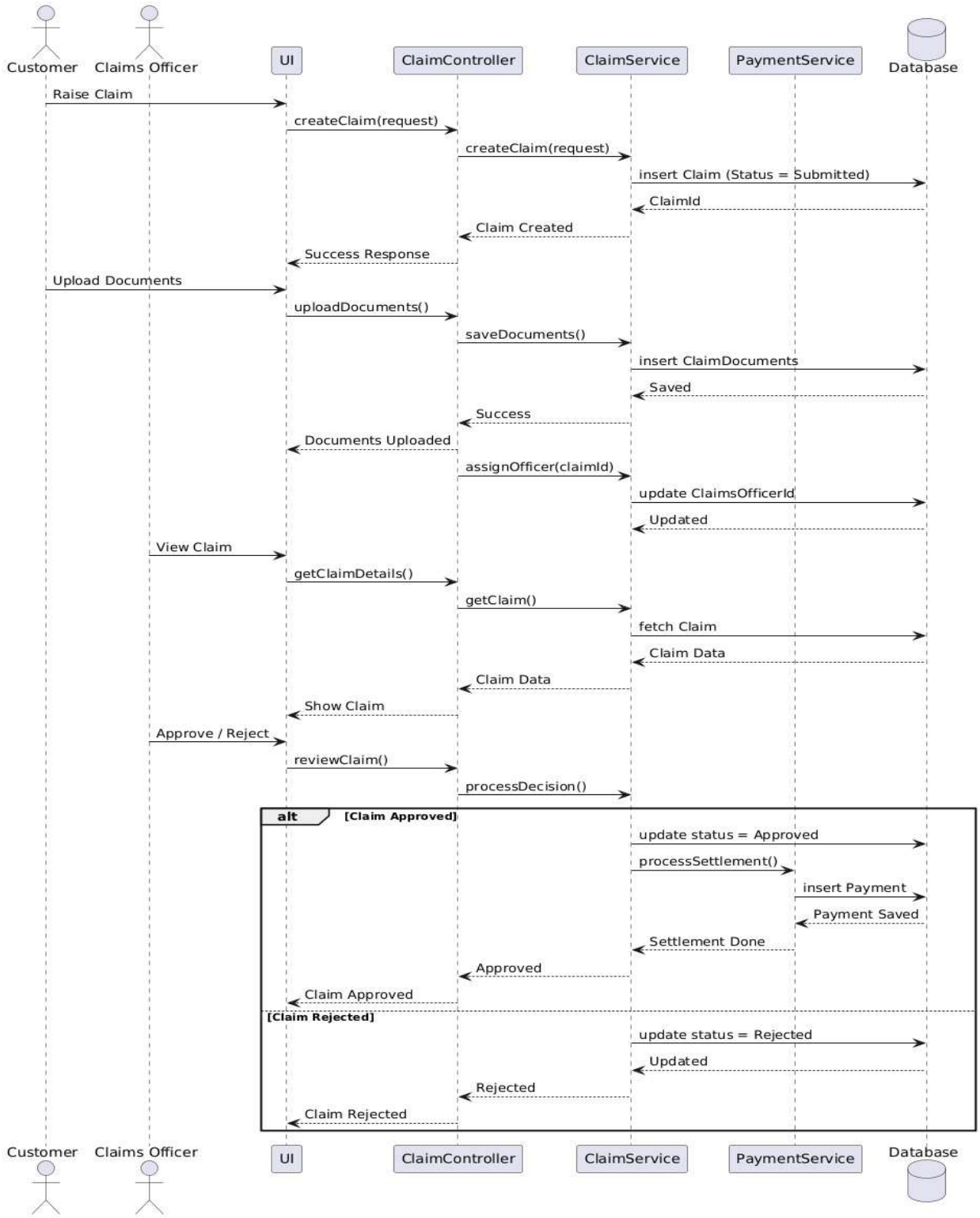

Figure 6: Claims Sequence Diagram showing the Claims Processing Flow

10. Technical Challenges & Solutions

Challenge	Problem	Solution
Multi-Role State Collisions	Managing claim state transitions across roles without data collisions. For example, a Surveyor trying to edit a claim that has already been handed off to the Claims Officer.	Strict State Machine logic enforced at the Service layer using Enum checks. Each transition is explicitly validated. Invalid transitions return 422.
Efficient File Upload Handling	Storing large image files as BLOBs directly in SQL Server severely degraded database performance.	Shifted to filesystem streaming. Files are written to secure local storage, and only the lightweight file paths are stored in the database.
PDF Generation in API Lifecycle	Integrating complex iText 9 cryptographic PDF generation within the synchronous API response lifecycle risked blocking server threads.	Abstracted PDF generation into a segregated background service that produces the stream asynchronously and returns a transient FileResult to the client.

11. Future Enhancements

Enhancement	Description	Priority
RTO / DMV API Integration	Automate vehicle data retrieval using official government registration APIs to prevent fraudulent entries and reduce input errors.	High
Real Payment Gateway	Replace simulated payments by integrating Stripe or PayPal webhooks for live payment processing and webhook-based status confirmation.	High
AI-Based Damage Assessment	Implement ML models with computer vision to automatically estimate vehicle damage from photos and flag potentially fraudulent claims before Surveyor review.	Medium
Automated Notifications	Integrate SMTP/SMS gateways (e.g. Twilio, SendGrid) via background workers to notify customers of policy and claim status updates in real time.	Medium
Policy Renewal Automation	Automatically generate renewal reminders and draft renewal policies before expiry using background scheduled jobs.	Medium
Mobile Application	Extend the platform to native iOS and Android apps for customer self-service, claim photo capture, and push notifications.	Low

12. Conclusion

This Software Requirements Specification has defined the complete functional, non-functional, architectural, and data requirements for ShieldDrive, the Vehicle Insurance Management System.

The system addresses real, measurable pain points in the traditional insurance industry: manual processing overhead, claim settlement delays, lack of transparency, and error-prone premium calculations. By replacing these with secure, automated, role-driven digital workflows, VIMS delivers concrete value to both insurance providers and policyholders.

The architecture, built on a Clean Architecture backend, a decoupled Angular frontend, JWT-based RBAC, Entity Framework ORM, and iText PDF generation, provides a solid, maintainable, and extensible foundation. The Vehicle Risk Score Engine, Premium Calculator Engine, and Claims Risk Engine together form a sophisticated automated underwriting and adjudication layer that removes human error from critical financial calculations.

With the future enhancements outlined in Section 11, VIMS is positioned to grow into a production-ready, AI-augmented insurance platform capable of integrating with government APIs, live payment gateways, and mobile clients.